

而那么做的成本又是多少。在对象之间搬移数值的最直接办法是通过赋值 (*assignment*) 操作, 但赋值的成本几何? 对许多 *types* 而言它相当于调用一个析构函数 (用以销毁旧值) 加上一个构造函数 (用以复制新值)。但你的目标是避免构造和析构成本耶! 面对现实吧, 这个做法不会成功的。就算以 *vector* 替换 *array* 也不会让情况更好些。

一个“必须返回新对象”的函数的正确写法是: 就让那个函数返回一个新对象呗。对 *Rational* 的 *operator** 而言意味以下写法 (或其他本质上等价的代码):

```
inline const Rational operator * (const Rational& lhs, const Rational& rhs)
{
    return Rational(lhs.n * rhs.n, lhs.d * rhs.d);
}
```

当然, 你得承受 *operator** 返回值的构造成本和析构成本, 然而长远来看那只是为了获得正确行为而付出的一个小小代价。但万一账单很恐怖, 你承受不起, 别忘了 C++ 和所有编程语言一样, 允许编译器实现者施行最优化, 用以改善产出码的效率却不改变其可观察的行为。因此某些情况下 *operator** 返回值的构造和析构可被安全地消除。如果编译器运用这一事实 (它们也往往如此), 你的程序将继续保持它们该有的行为, 而执行起来又比预期的更快。

我把以上的讨论浓缩总结为: 当你必须在“返回一个 *reference* 和返回一个 *object*”之间抉择时, 你的工作就是挑出行为正确的那个。就让编译器厂商为“尽可能降低成本”鞠躬尽瘁吧, 你可以享受你的生活。

请记住

- 绝不要返回 *pointer* 或 *reference* 指向一个 *local stack* 对象, 或返回 *reference* 指向一个 *heap-allocated* 对象, 或返回 *pointer* 或 *reference* 指向一个 *local static* 对象而有可能同时需要多个这样的对象。条款 4 已经为“在单线程环境中合理返回 *reference* 指向一个 *local static* 对象”提供了一份设计实例。

条款 22: 将成员变量声明为 *private*

Declare data members *private*.

OK, 下面是我的规划。首先带你看看为什么成员变量不该是 *public*, 然后让你看看所有反对 *public* 成员变量的论点同样适用于 *protected* 成员变量。最后导出一个结论: 成员变量应该是 *private*。获得这个结论后, 本条款也就大功告成了。

好, 现在看看 `public` 成员变量。为什么不采用它呢?

让我们从语法一致性开始 (同时请见条款 18)。如果成员变量不是 `public`, 客户唯一能够访问对象的办法就是通过成员函数。如果 `public` 接口内的每样东西都是函数, 客户就不需要在打算访问 `class` 成员时迷惑地试着记住是否该使用小括号 (圆括号)。他们只要做就是了, 因为每样东西都是函数。就生命而言, 这至少可以省下许多搔首弄耳的时间。

或许你不认为一致性的理由足以令人信服, 那么这个事实如何: 使用函数可以让你对成员变量的处理有更精确的控制。如果你令成员变量为 `public`, 每个人都可以读写它, 但如果你以函数取得或设定其值, 你就可以实现出“不准访问”、“只读访问”以及“读写访问”。见鬼了, 你甚至可以实现“惟写访问”, 如果你想要的话:

```
class AccessLevels {
public:
    ...
    int getReadOnly() const { return readOnly; }
    void setReadWrite(int value) { readWrite = value; }
    int getReadWrite() const { return readWrite; }
    void setWriteOnly(int value) { writeOnly = value; }
private:
    int noAccess;           //对此 int 无任何访问动作
    int readOnly;           //对此 int 做只读访问 (read-only access)
    int readWrite;          //对此 int 做读写访问 (read-write access)
    int writeOnly;          //对此 int 做惟写访问 (write-only access)
};
```

如此细微地划分访问控制颇有必要, 因为许多成员变量应该被隐藏起来。每个成员变量都需要一个 `getter` 函数和 `setter` 函数毕竟罕见。

还是不够说服你? 是端出大口径武器的时候了: 封装啦。如果你通过函数访问成员变量, 日后可改以某个计算替换这个成员变量, 而 `class` 客户一点也不会知道 `class` 的内部实现已经起了变化。

举个例子，假设你正在写一个自动测速程序，当汽车通过，其速度便被计算并填入一个速度收集器内：

```
class SpeedDataCollection {  
    ...  
public:  
    void addValue(int speed);           //添加一笔新数据  
    double averageSoFar() const;       //返回平均速度  
    ...  
};
```

现在让我们考虑成员函数 `averageSoFar`。做法之一是在 `class` 内设计一个成员变量，记录至今以来所有速度的平均值。当 `averageSoFar` 被调用，只需返回那个成员变量就好。另一个做法是令 `averageSoFar` 每次被调用时重新计算平均值，此函数有权力调取收集器内的每一笔速度值。

上述第一种做法（随时保持平均值）会使每一个 `SpeedDataCollection` 对象变大，因为你必须为用来存放目前平均值、累积总量、数据点数的每一个成员变量分配空间。然而 `averageSoFar` 却可因此而十分高效；它可以只是一个返回目前平均值的 `inline` 函数（见条款 30）。相反地，“被询问才计算平均值”会使得 `averageSoFar` 执行较慢，但每一个 `SpeedDataCollection` 对象比较小。

谁说得出哪一个比较好？在一部内存吃紧的机器上（例如一台嵌入式路边侦测装置），或是在一个并不常常需要平均值的应用程序中，“每次需要时才计算”或许是比较好的解法。但在一个频繁需要平均值的应用程序中，如果反应速度非常重要，内存不是重点，这时候“随时维持一个当下平均值”往往更好一些。重点是，由于通过成员函数来访问平均值（也就是封装了它），你得以替换不同的实现方式（以及其他你可能想到的东西），客户最多只需重新编译。（如果遵循条款 31 所描述的技术，你甚至可以消除重新编译的不便性。）

将成员变量隐藏在函数接口的背后，可以为“所有可能的实现”提供弹性。例如这可使得成员变量被读或被写时轻松通知其他对象、可以验证 `class` 的约束条件以及函

数的前提和事后状态、可以在多线程环境中执行同步控制……等等。来自 Delphi 和 C# 阵营的 C++ 程序员应该知道, 这般能力等价于其他语言中的 "properties", 尽管额外需要一组小括号。

封装的重要性比你最初见到它时还重要。如果你对客户隐藏成员变量 (也就是封装它们), 你可以确保 `class` 的约束条件总是会获得维护, 因为只有成员函数可以影响它们。犹有进者, 你保留了日后变更实现的权利。如果你不隐藏它们, 你很快会发现, 即使拥有 `class` 原始码, 改变任何 `public` 事物的能力还是极端受到束缚, 因为那会破坏太多客户码。`Public` 意味不封装, 而几乎可以说, 不封装意味不可改变, 特别是对被广泛使用的 `classes` 而言。被广泛使用的 `classes` 是最需要封装的一个族群, 因为它们最能够从“改采用一个较佳实现版本”中获益。

`protected` 成员变量的论点十分类似。实际上它和 `public` 成员变量的论点相同, 虽然或许最初看起来不是一回事。“语法一致性”和“细微划分之访问控制”等理由显然也适用于 `protected` 数据, 就像对 `public` 一样适用。但封装呢? `protected` 成员变量的封装性是不是高过 `public` 成员变量? 答案令人惊讶: 并非如此。

条款 23 会告诉你, 某些东西的封装性与“当其内容改变时可能造成的代码破坏量”成反比。因此, 成员变量的封装性与“成员变量的内容改变时所破坏的代码数量”成反比。所谓改变, 也许是从 `class` 中移除它 (或许这有利于计算, 就像上述的 `averageSoFar`)。

假设我们有一个 `public` 成员变量, 而我们最终取消了它。多少代码可能会被破坏呢? 唔, 所有使用它的客户码都会被破坏, 而那是一个不可知的大量。因此 `public` 成员变量完全没有封装性。假设我们有一个 `protected` 成员变量, 而我们最终取消了它, 有多少代码被破坏? 唔, 所有使用它的 `derived classes` 都会被破坏, 那往往也是个不可知的大量。因此, `protected` 成员变量就像 `public` 成员变量一样缺乏封装性, 因为在这两种情况下, 如果成员变量被改变, 都会有不可预知的大量代码受到破坏。虽然这个结论有点违反直观, 但经验丰富的程序库作者会告诉你, 它是真的。一旦你将一个成员变量声明为 `public` 或 `protected` 而客户开始使用它, 就很难改变那个成员变量所涉及的一切。太多代码需要重写、重新测试、重新编写文档、重新编译。从封装的角度观